CNENGR202 Instrumentation and Control

Part 3 L4 – Delays and timers

Prof. Zhang Liwei 张立伟 教授

Email: lwzhang@bjtu.edu.cn
Working Room: SX702

School of Electronics and Information Engineering
Beijing Jiaotong University

Recap

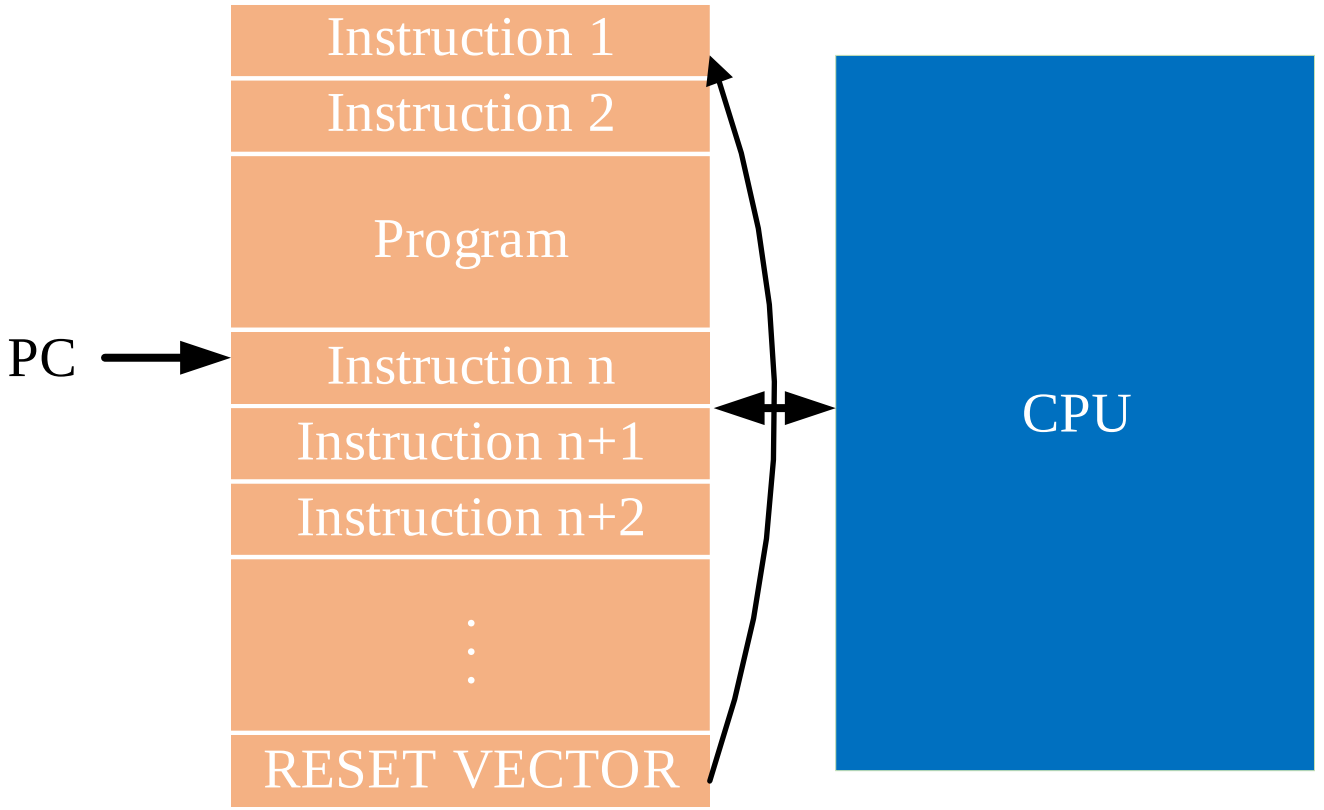
- Programming in Arduino
  - Structure
  - Variables
  - Functions
- Example codes

This lecture …

- We will see how for loops and conditional statements are actually implemented
- Delays
- Timers

What actually happens - I

- Program loops anyway
- When you get to the end it starts again



What actually happens - II

```
label:
        NOP // do nothing
        RJMP label
        LDI r16, 0b0 // never reached
```

- This will make the processor do nothing (NOP) repeatedly, forever
- The final line is never reached
- Stuck in the loop forever!

What actually happens - III

```
Loop:
    INC r1
    RJMP Loop
Finished:
    //do other stuff
```

- We need to branch out of the loop!
- Branching allows your programme to make decisions based on the result of the previous command.

What actually happens - IV

- To understand branching we need to understand the status register
- The **status register** is show the status of the instruction just executed
- Example: Show number overflow
  - Almost everything is 8bit
    - numbers are limited in size
  - What happens when we run out of space?
  - Add 1 to 255
    - Equivalent of 0b00000001 + 0b11111111
  - Answer is 0b00000000 or 0

What actually happens - V

- Status REGister contains bits that indicate the results of the instruction just executed

Bit	7	6	5	4	3	2	1	0
	I	T	H	S	V	N	Z	C

SREG

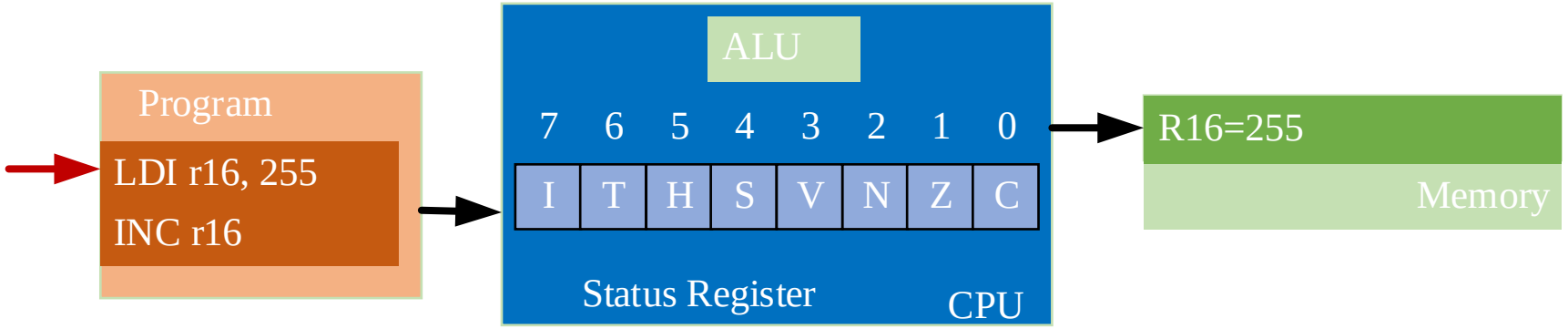
Add 1 to 255 →

Bit 7 – I: Global Interrupt Enable
Bit 6 – T: Bit Copy Storage
Bit 5 – H: Half Carry Flag
Bit 4 – S: Sign Bit, S = NV
Bit 3 – V: Two's Complement Overflow Flag
Bit 2 – N: Negative Flag
Bit 1 – Z: Zero Flag
Bit 0 – C: Carry Flag



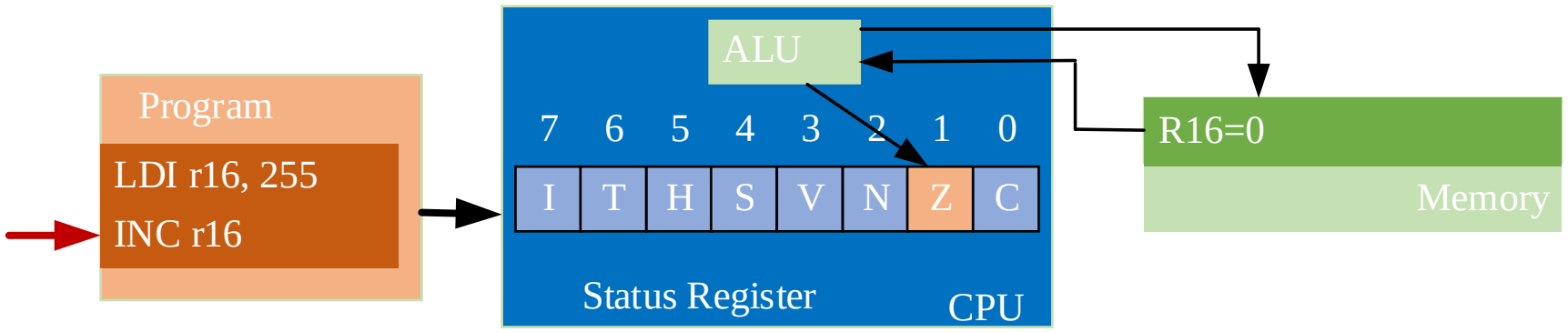
INC - I

- Setup r16



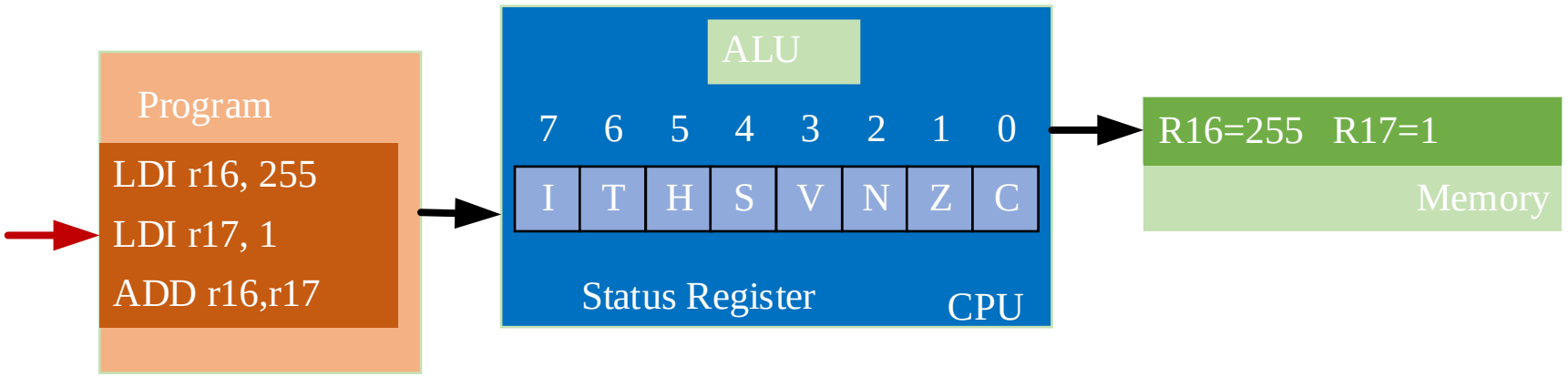
INC - II

- Increment r16
- Update status register



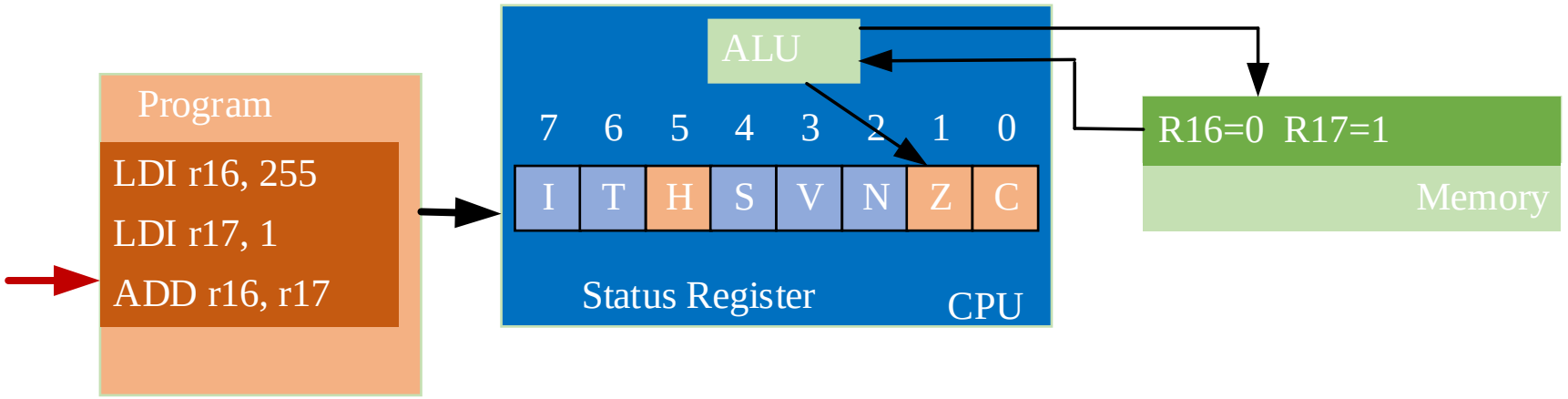
ADD - I

- Setup r16 and r17



ADD - II

- Add r16 and r17
- Update status register



Branching - I

- Why is this useful?
- BREQ Label – branch if Zero Flag is equal to 1
- BRNE Label – branch if Zero Flag is equal to 0
- So now we have

```
Loop:
    INC r1
    BREQ Finished
    RJMP Loop
Finished:
    LDI r16, 34 //do something
```

Branching - II

- Also can be based on comparison (>=, >, <, <=, ==)if preceded by the *CP* (compare) command

```
        LDI r25, 10
Loop:
        LDI r1, 6
        CP r1,r25
        BRGE Finished
        RJMP Loop
Finished:
        LDI r16,5 // do something
```

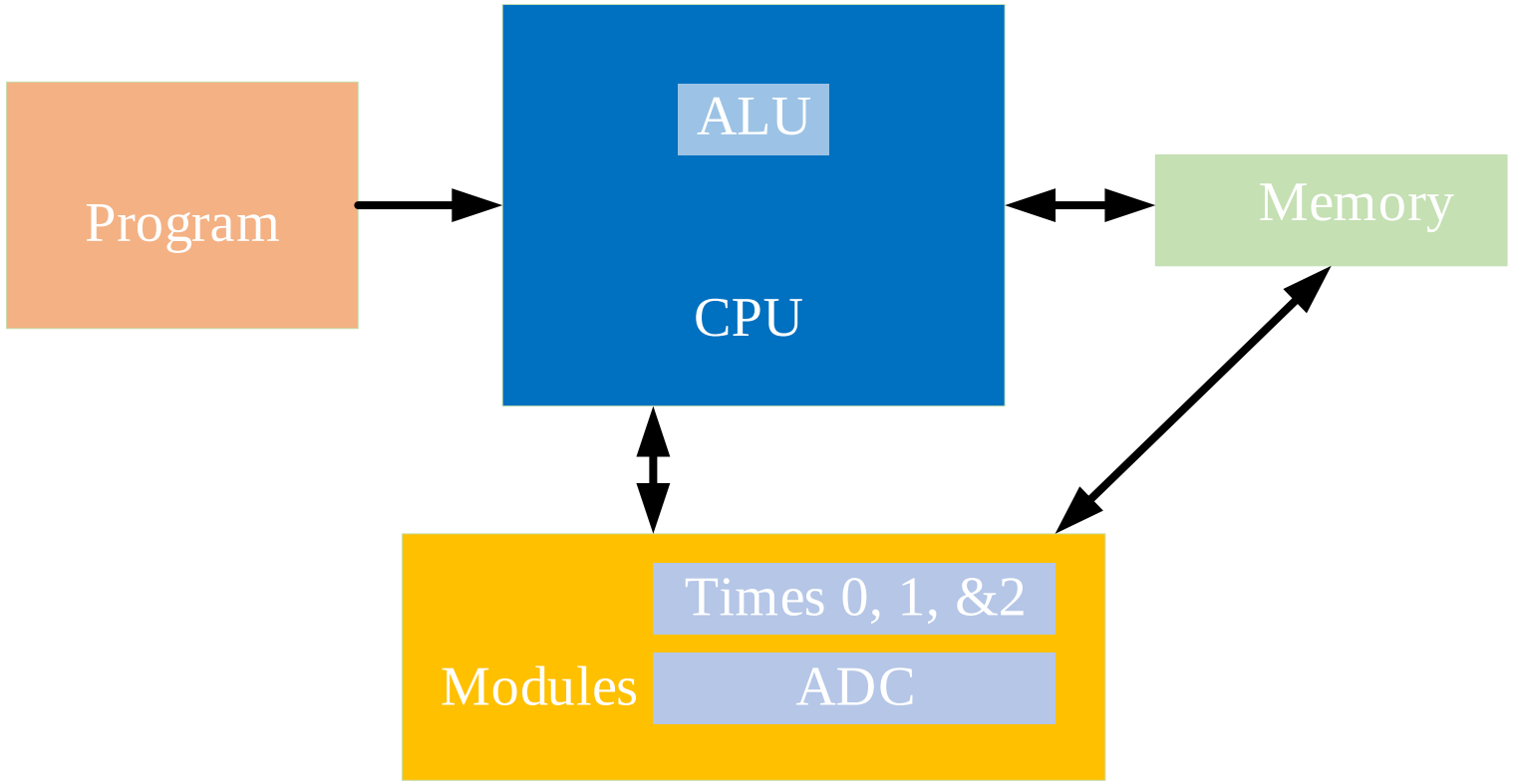
Delays

- Chip runs at 16MHz, loops run very fast
- Normally want a more human timescale
- Loops can make our program stop for a while
- It is possible to calculate exactly how long the code will take to execute.
- Most language commands (assembly) are a single cycle.

Time accuracy

- Hard to hit a specific delay, because the values held in register are:
  - Restricted to integers.
- A difference of milliseconds
  - no consequence to washing machines
  - ABS system would only correlate to ~8 mm at 100 km/h
  - data transmission – out of sync
- This technique wastes CPU time and energy

Modules



What the timer does

- Main functions
  - A simply counter – independent to program execution
  - Pulse Width Modulation (PWM)
  - Input waveform capture
- Fundamental to implement:
  - Time delay creation and measurement
  - Period and pulse width measurement
  - Frequency measurement
  - Event counting
  - Arrival time comparison
  - Time - of - day tracking
  - Periodic interrupt generation
  - Waveform generation.

Alternatives to loops – in - built timers

- Most microcontrollers have built - in timers.
- The device we are using has three, referred to as Timer0, Timer1 and Timer2.
- It's easier to understand if you think of them as a Counter
- Count from **0** to **255** then set an overflow bit

Free running counter

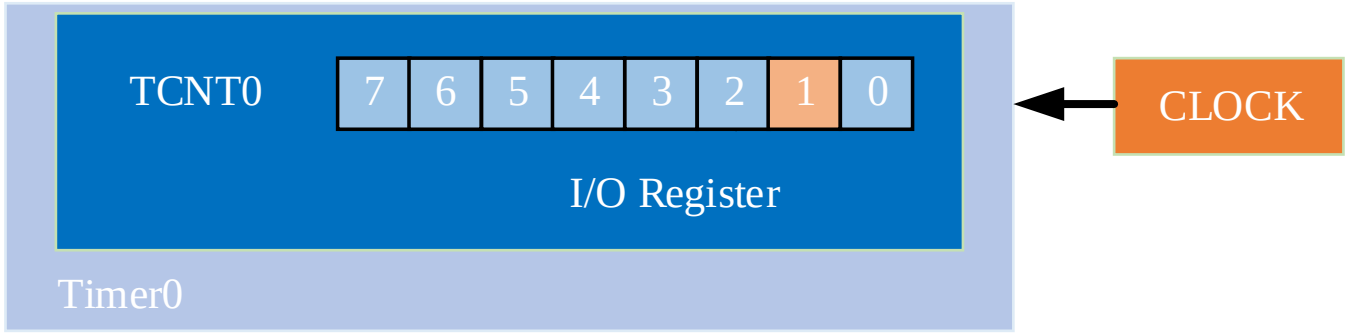
- ***Free - running counter is a simple register that is incremented by 1 on each clock tick.***
- The value in the timer at a specific moment in time is held in the counter register:
  - *TCNT0*
  - We can watch this in the I/O view of AtmelStudio
- The counter counts from 00 to 0xFF and then rolls back over to 00.

Timer0

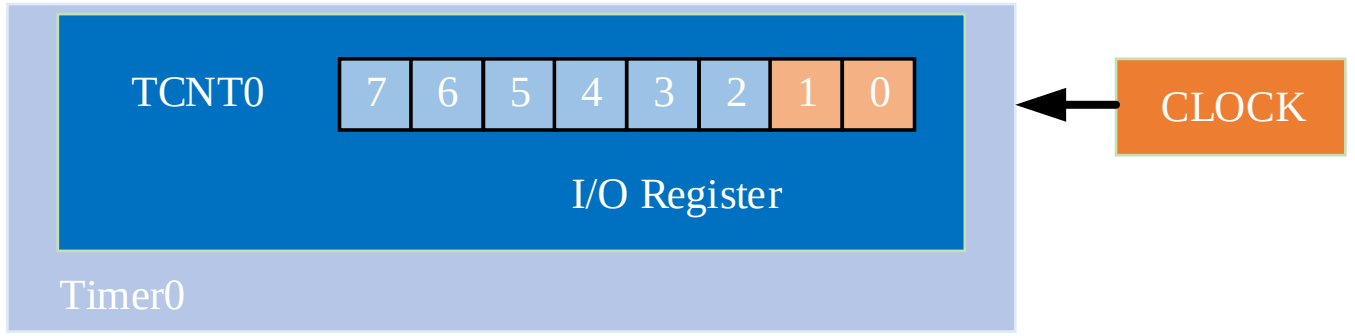
1 clock



2 clock

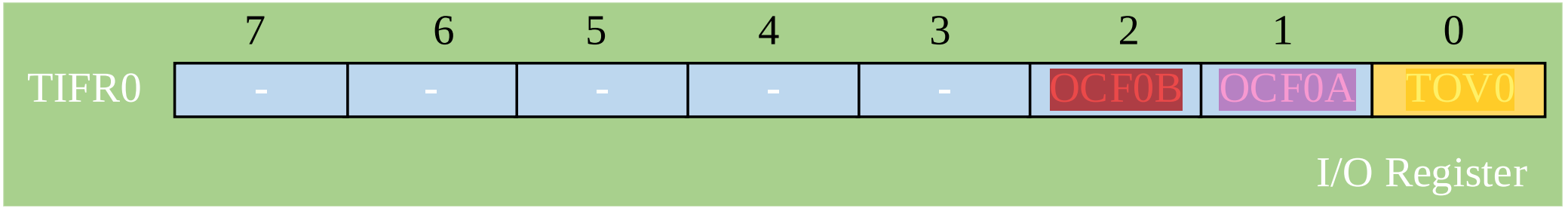


3 clock



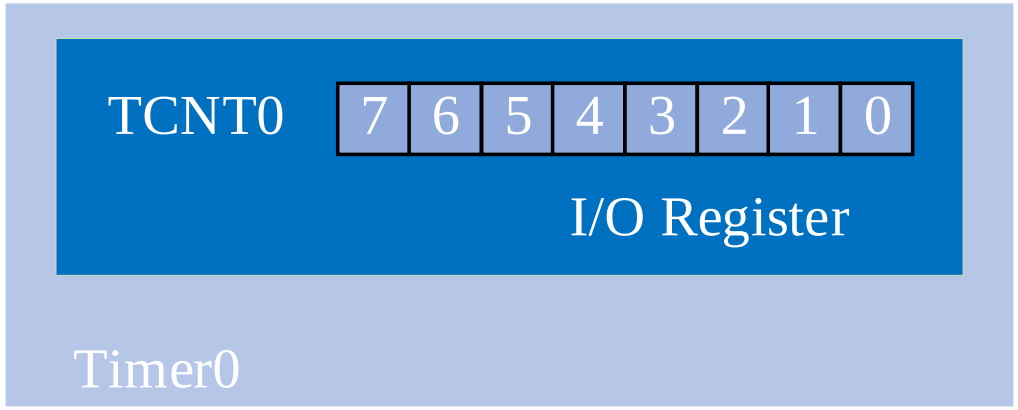
Status Bit

- This is known as an overflow, this sets the 0th bit of a status register TIFR0 (Timer Interrupt Flag Register) TIFR0---定时器中断标志寄存器
- This bit is called TOV0 (Timer OVerflow for timer 0)
- Program responsible for resetting to 0 by writing a 1

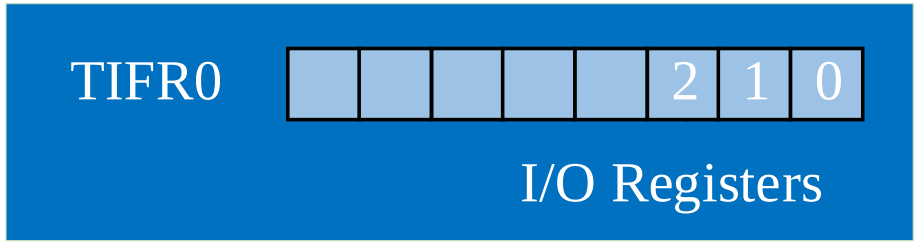


Overflow - I

- Reach maximum value

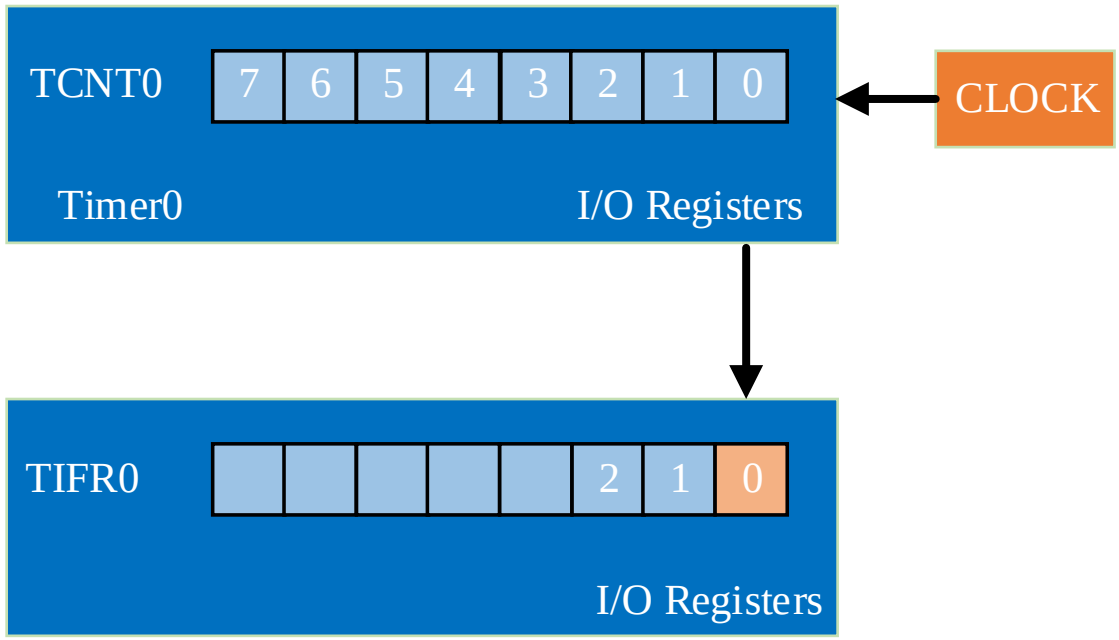


CLOCK



Overflow - II

- Overflow back to 0
- Set TOV0



How is delay implemented?

```
        LDI  r16,0b0000_0001
        OUT  TCCR0B, R16
MAIN_LOOP:                          ;start the timer
        RCALL  PAUSE                ;call pause routine (we will come back here on ret)
        do  something
        RJMP  MAIN_LOOP             ;start again

PAUSE:  IN  r16,TIFR0               ;load the status register
        ANDI  r16,0b0000_0001       ;checks the 0th bit to see if counter has overflowed
        BREQ  PAUSE                 ;if answer of previous line is zero (not overflowed)
        LDI   r16,0b0000_0001       ;check again

        OUT   TIFR0,r16             ;reset the flag (reference SBI PINB)
        RET                         ;return to main loop
```

Speed

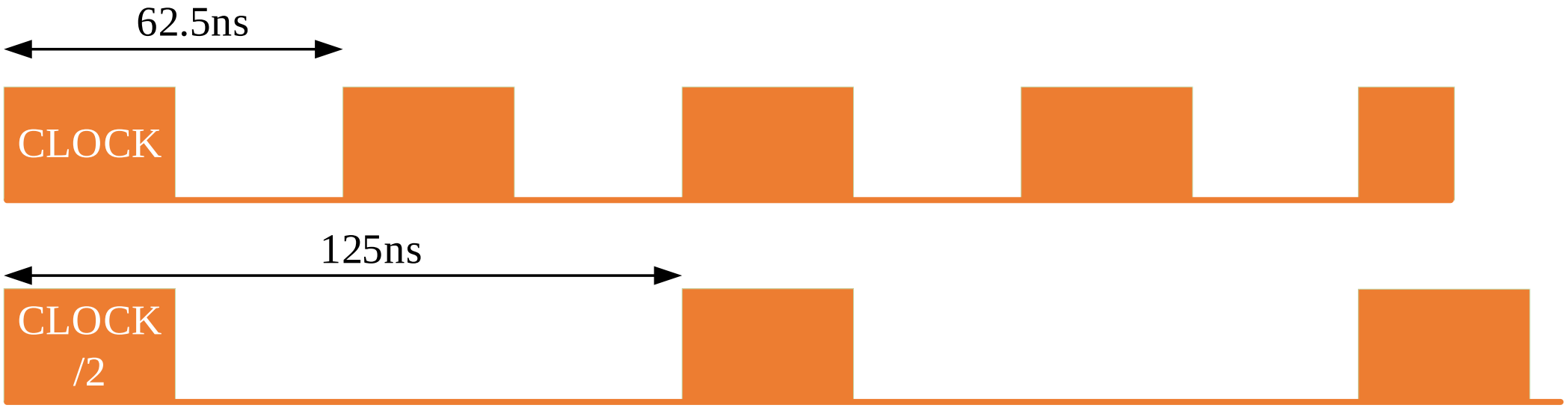
- Bus clock = 16 MHz (62.5 nanoseconds)
- The counter 8 bits wide so they only have a span of 255 clock cycles or 256*62.5=16 microseconds

- Well that's **fast**!
- Failing at human timescales again.

The Timer0 counter - Prescaler

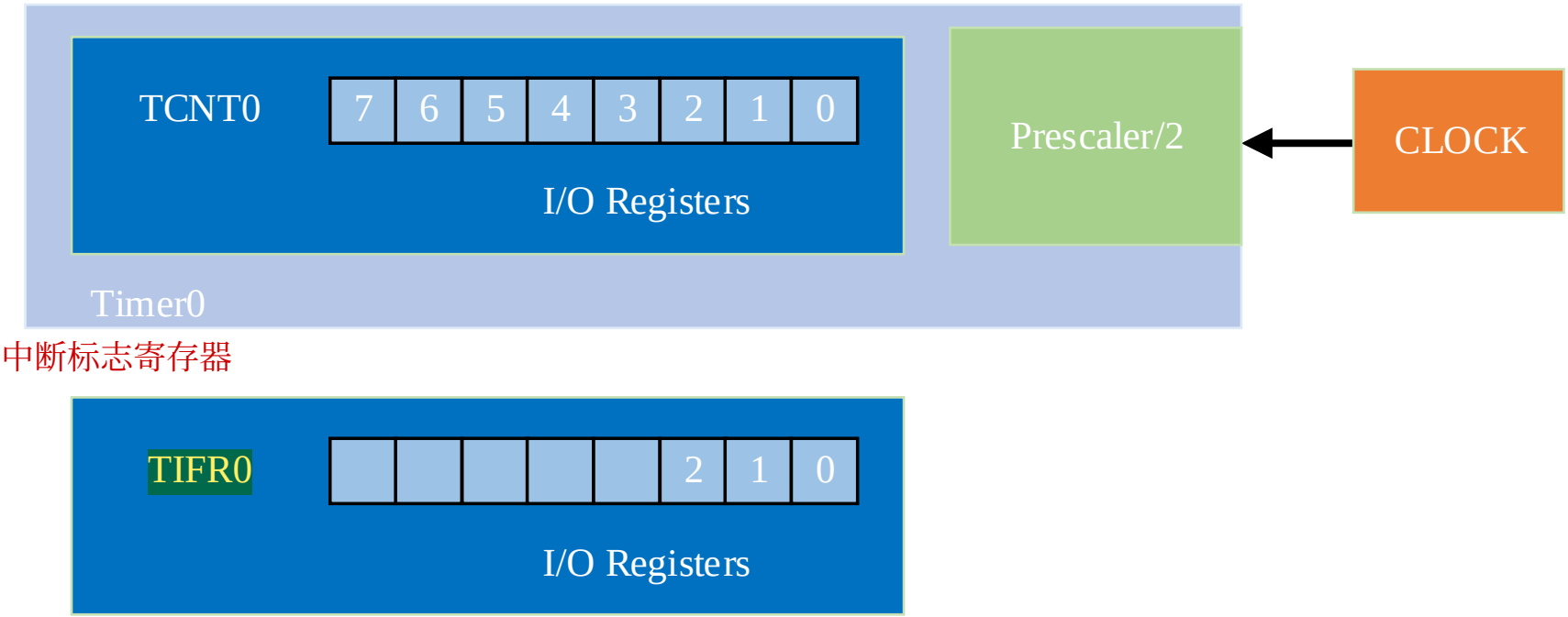
- The input clock rate can be slowed down using the prescaler before feeding to the timer
- This divides the clock rate by a fixed amount

- **Slow down the counter**



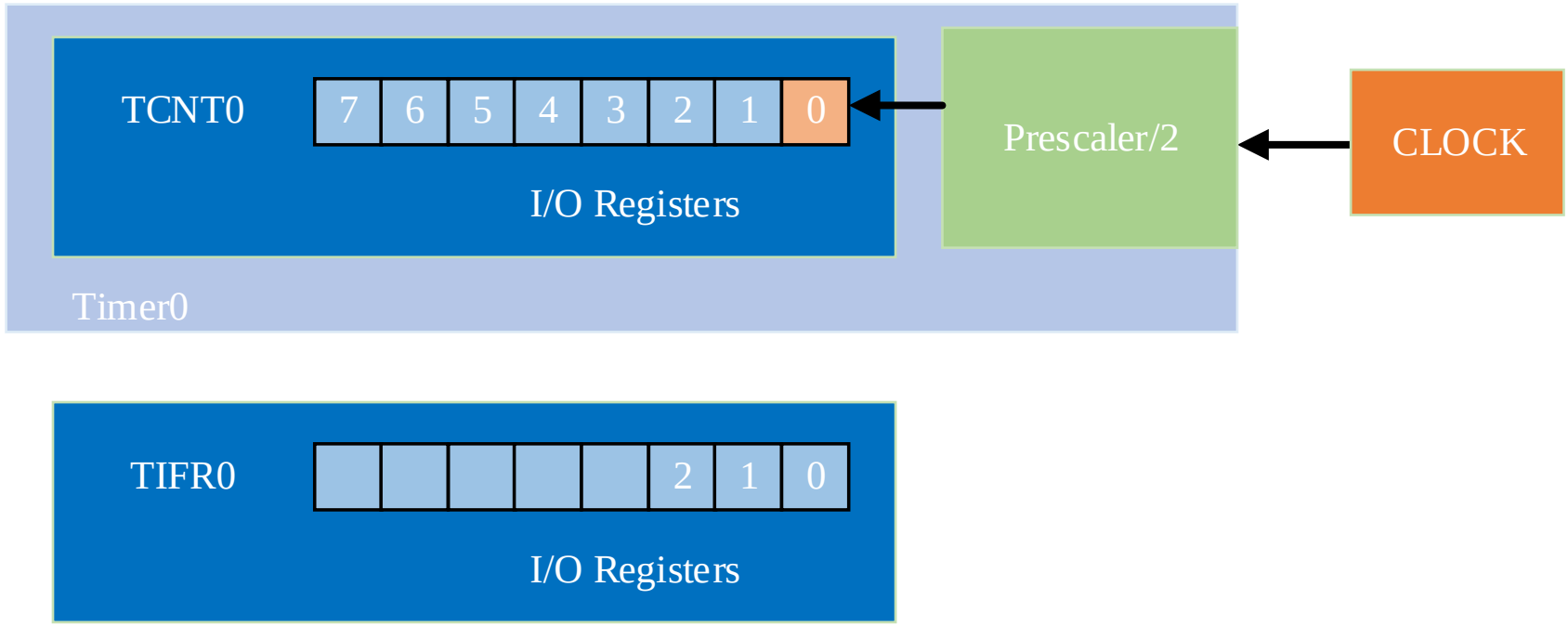
Timer0 - I

- Prescale of 2 is not available BUT it's easier to see what's going on

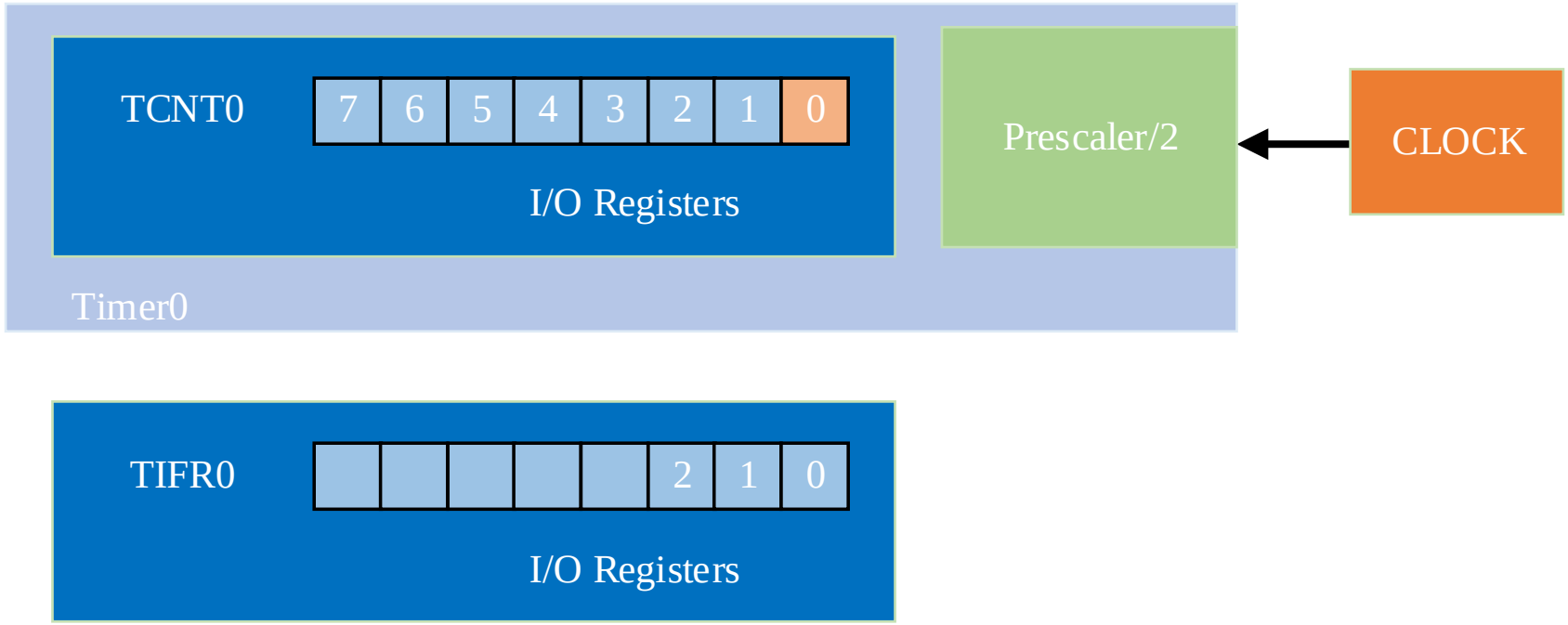


TIFR0---定时器中断标志寄存器

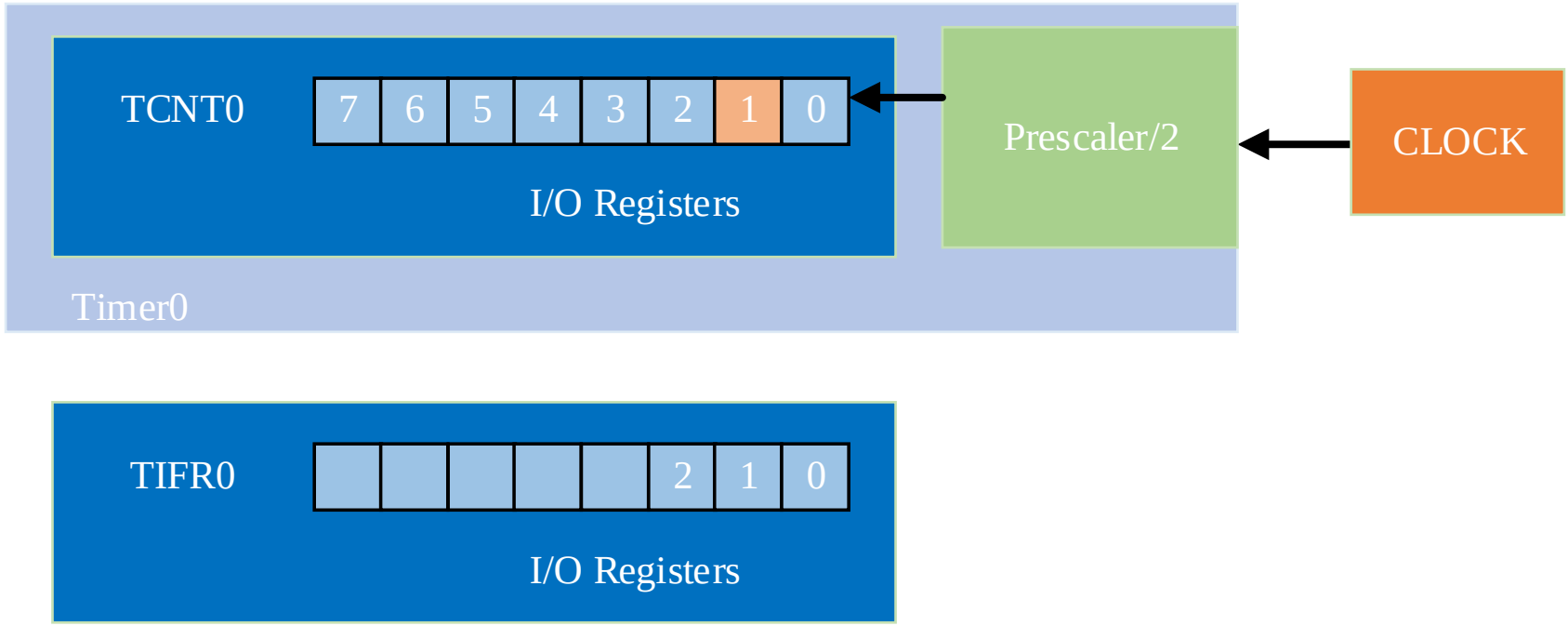
Timer0 - II


TCNT0
7
6
5
4
3
2
1
0
I/O Registers
Prescaler/2
CLOCK
Timer0
TIFR0
2
1
0
I/O Registers

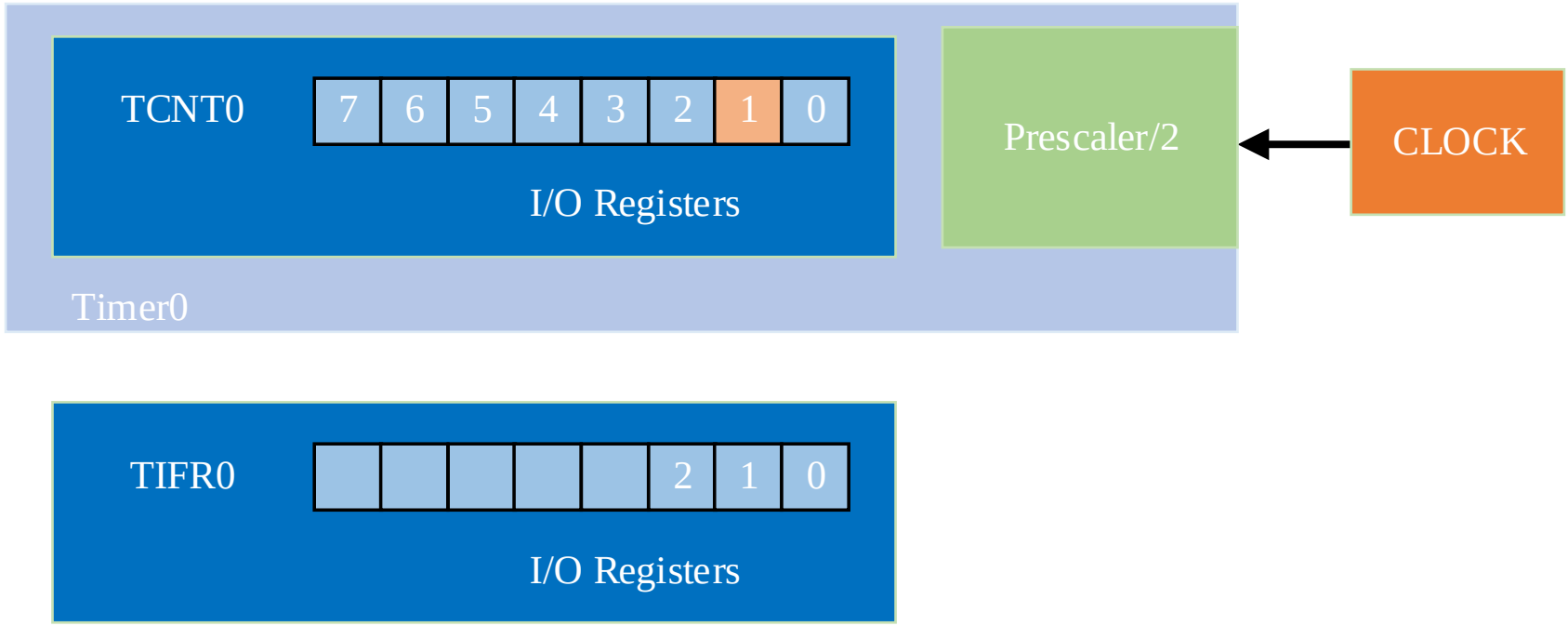
Timer0 - III


TCNT0
7
6
5
4
3
2
1
0
I/O Registers
Prescaler/2
CLOCK
Timer0
TIFR0
2
1
0
I/O Registers

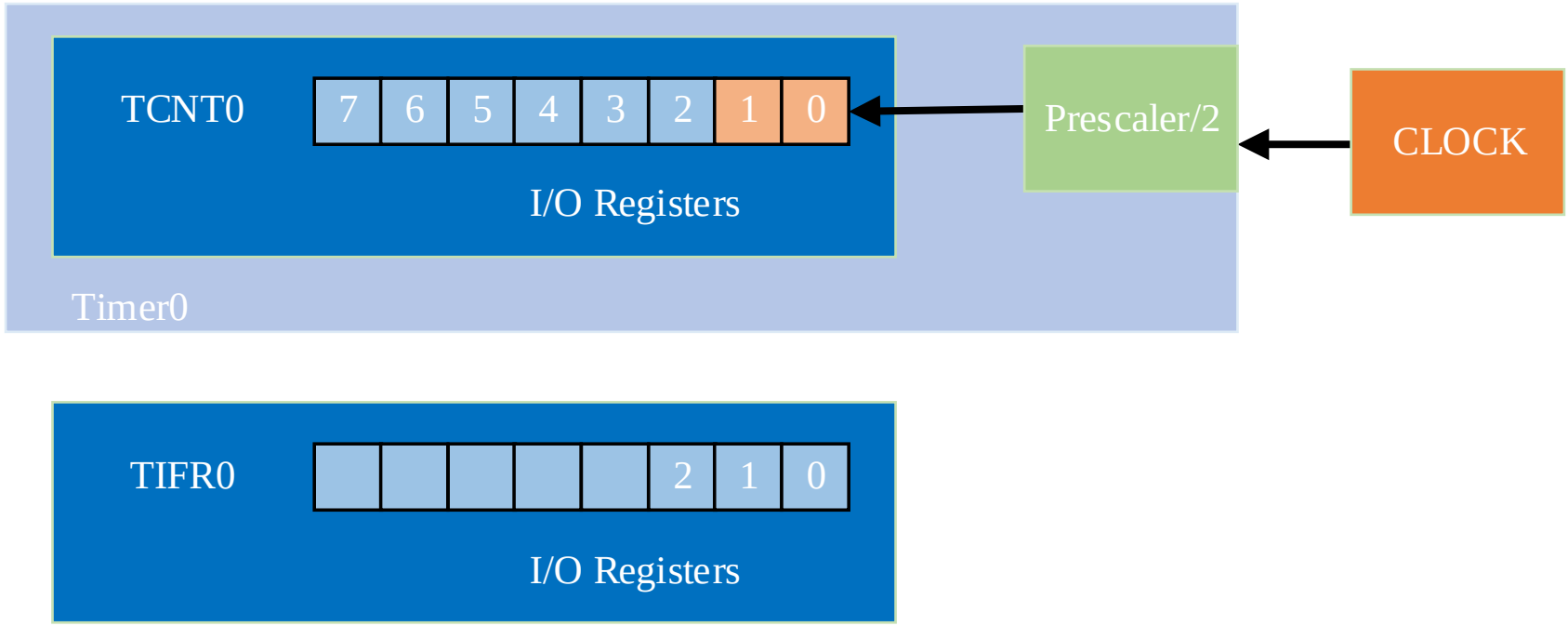
Timer0 _ IV


TCNT0
7
6
5
4
3
2
1
0
I/O Registers
Prescaler/2
CLOCK
Timer0
TIFR0
2
1
0
I/O Registers

Timer0 - V

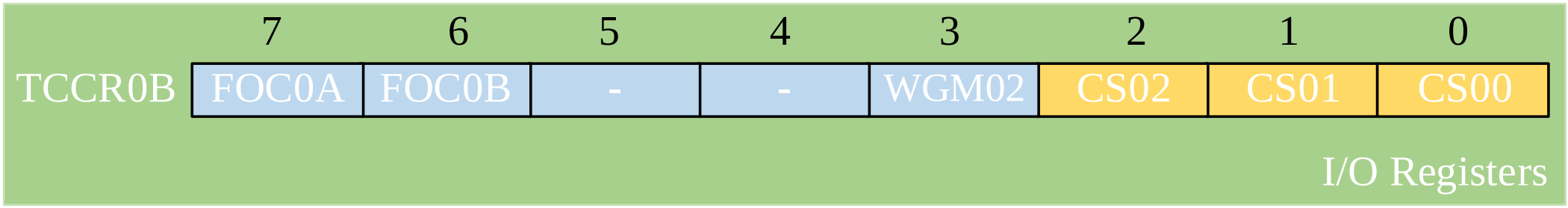

TCNT0
7
6
5
4
3
2
1
0
I/O Registers
Prescaler/2
CLOCK
Timer0
TIFR0
2
1
0
I/O Registers

Timer0 - VI


TCNT0
7
6
5
4
3
2
1
0
I/O Registers
Prescaler/2
CLOCK
Timer0
TIFR0
2
1
0
I/O Registers

Timer Counter Control Register B

- Prescaler controlled with CS0x bits in “port” TCCR0B
  - 3bit Clock divider

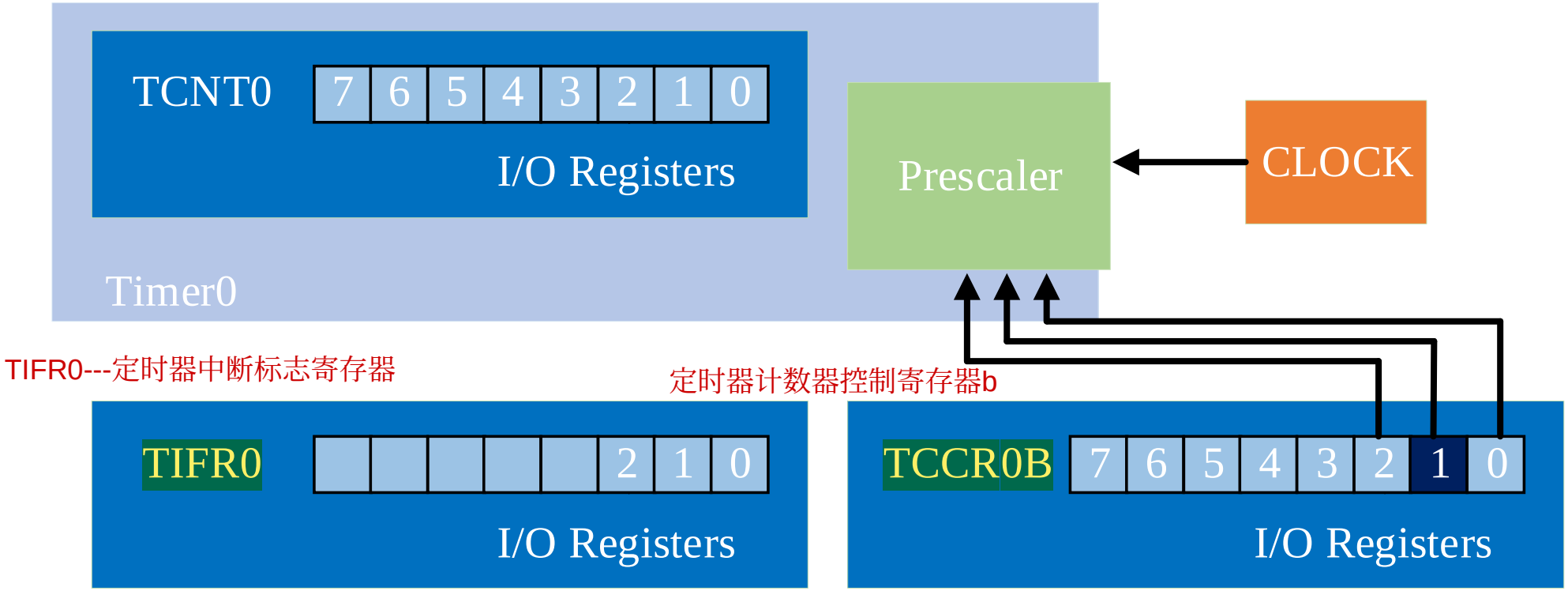


CS02	CS01	CS00	Description
0	0	0	Counter stopped
0	0	1	/1
0	1	0	/8
0	1	1	/64
1	0	0	/256
1	0	1	/1024

```
LDI A,0b0000_0101    ;SET TIMER PRESCALER TO 5 which means /1024

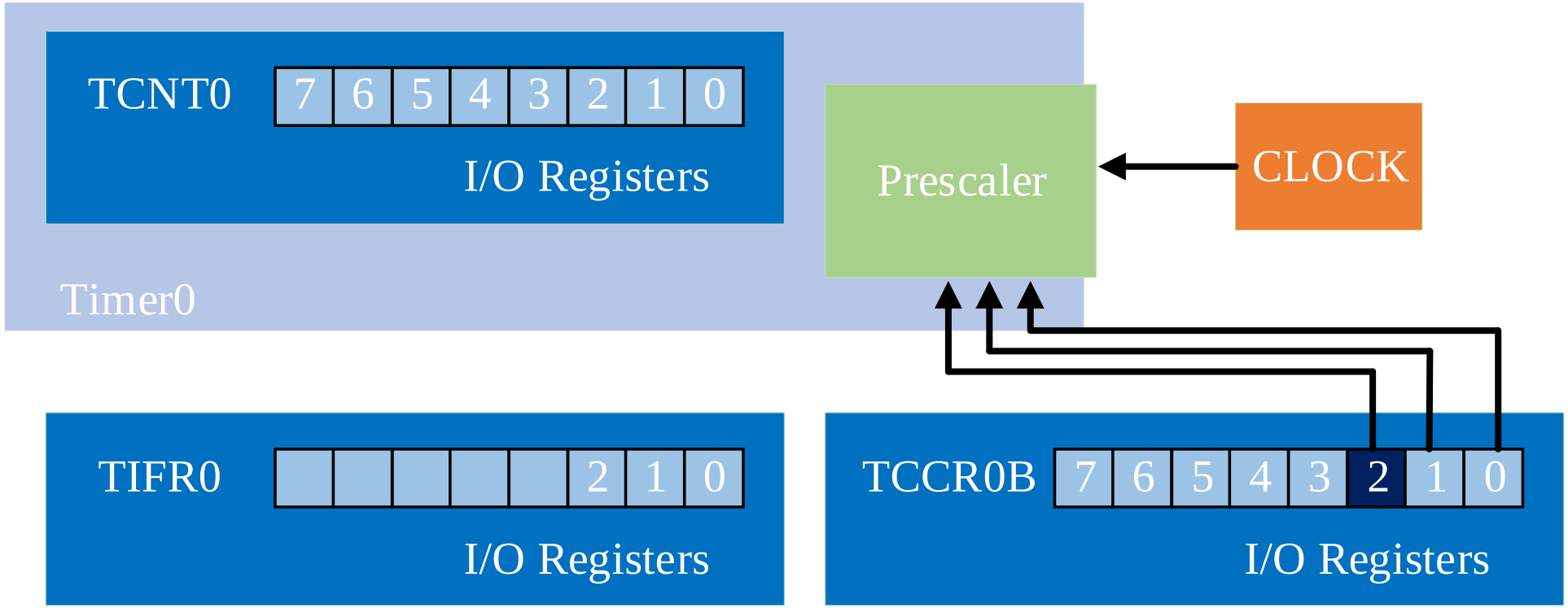
OUT TCCR0B,A         ;set prescaler in Timer/Counter Control Register (TCCR0B)
```

Timer0 - I



CS02	CS01	CS00	Description
0	1	0	/8

Timer0 - II



CS02	CS01	CS00	Description
1	0	0	/256

Prescale the timer

```
.def A = R16                ;define a register
LDI A,0b0000_0101           ;set prescaller to /1024
OUT TCCR0B,A                ;start the timer
MAIN_LOOP:
RCALL PAUSE                 ;call pause routine (we will come
do something                ;back here on ret)
RJMP MAIN_LOOP
PAUSE: IN  A,TIFR0          ;start again
ANDI  A,0b0000_0001         ;load the status register
BREQ  PAUSE                 ;checks the 0th bit to see if counter has overflowed
LDI   A,0b0000_0001         ;if answer of previous line is zero (not overflowed) check again
OUT   TIFR0, A              ;reset the flag
RET                         ;return to main loop
```

CS 02	CS 01	CS 00	Description
0	0	0	Counter stopped
0	0	1	/1
0	1	0	/8
0	1	1	/64
1	0	0	/256
1	0	1	/1024

Different timings

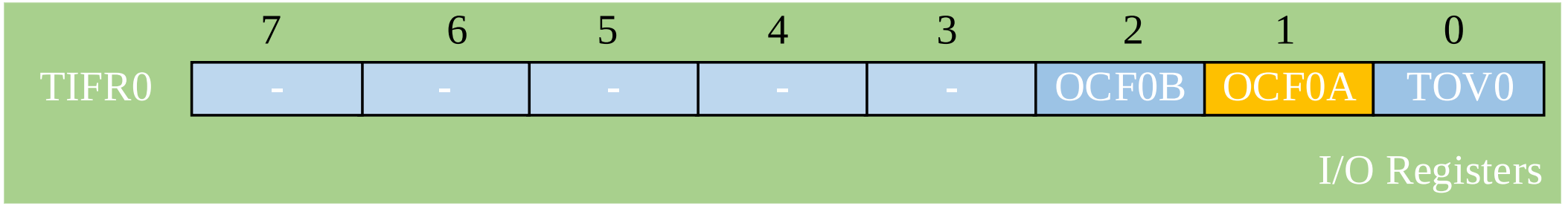
- Prescaling is good BUT only 5 options for timing (we can do better!).
- Load a value into *TCNT0*
- To count FROM 128 (rather than 0) to 255

```
LDI   A,128        ;PRELOAD TIMER WITH 128
OUT   TCNT0,A      ;
```

Change max

- Change the maximum value of the count.
  - overflow at this new value
- Value defined by a register called *OCR0A* *(output compare register A)*

- Now you want to check a different bit in *TIFR0*, the compare bit.
- This bit is set when *OCR0A* and *TCNT0 are equal*



Next Lecture

- Reacting to stimulus!